

3-2 装入与链接

• 程序生成与运行过程

• 编译与链接

- `main.c` : 源程序 (文本)
- 预处理器 `cpp`
 - 生成 `main.i`
 - 得到修改后的源程序 (文本)
- 编译器 `ccl`
 - 生成 `main.s`

 - 得到汇编程序 (文本)
- 汇编器 `as`
 - 生成 `main.o`
 - 得到可重定位目标程序 (二进制)
- 链接器 `ld`
 - 将 `main.o`、`sum.o` 等目标模块与所需库函数链接
 - 生成可执行目标程序 `main` (二进制)
- 装入程序
 - 将程序装入内存
 - 装入完成后, 由 CPU 执行



后发先至 抢占先机

• 一、链接

• 链接解决的问题

- 链接对象
 - 源程序编译后形成的目标模块
 - 目标模块调用的外部模块
 - 例如: 库函数
- 链接作用
 - 处理模块之间的外部引用
 - 将运行所需的模块组合起来
- 三种链接方式的区别
 - 核心在于: 什么时候链接
 - 不在于: 程序采用哪种内存管理方式

• 静态链接

- 链接时机
 - 程序装入内存之前
- 过程
 - 编译生成多个目标模块
 - 找到目标模块调用的外部模块
 - 将各目标模块、外部模块和库函数全部链接
 - 形成一个完整的装入模块
 - 再将装入模块装入内存

- 特点

- 链接完成后不再拆开
- 运行时不需要再次链接



只要目标模块中存在外部调用，对应模块通常就会提前链接



- 即使某段代码实际没有执行，对应外部模块也可能已被链接、装入

- **装入时动态链接**

- 链接时机
 - 目标模块装入内存时
- 过程
 - 编译后得到一组目标模块
 - 将目标模块一个一个装入内存
 - 装入过程中检查外部模块调用
 - 找到对应外部模块
 - 边装入、边链接
- 特点
 - 链接推迟到装入阶段
 - 各目标模块可分别存放
 - 只要在目标模块中发现调用，装入时就会链接相应外部模块
- 与静态链接的区别
 - 静态链接：装入前一次性链接完成
 - 装入时动态链接：装入过程中逐个链接

后浪科技公众号ID: 1931057888

- **运行时动态链接**

- 链接时机
 - 程序装入内存后
 - CPU 真正执行到外部模块调用时
- 过程
 - 程序开始运行

- CPU 执行到某个外部模块或库函数
- 操作系统查找对应模块
- 将该模块调入内存
- 与当前目标模块完成链接
- CPU 继续执行

- 特点

- 按需调入、按需链接
- 未被实际执行的外部模块不装入、也不链接
- 可以避免无用模块占用内存
- 内存利用率更高

- 典型例子



程序中包含错误处理模块

- 若运行时没有发生错误
- 错误处理模块就不需要调入和链接

- 考试提示

- 题目只说“动态链接”时
- 通常默认指运行时动态链接

后浪科技公众号ID: 1931057098



- **三种链接方式对比**

- 静态链接

- 时机：装入之前
- 做法：提前链接成完整装入模块
- 外部模块：可能全部提前装入

- 装入时动态链接

- 时机：装入内存时
- 做法：边装入、边链接
- 外部模块：发现调用后，在装入阶段链接

- 运行时动态链接

- 时机：真正运行时
- 做法：执行到调用位置时再链接
- 外部模块：实际需要时才调入

- **链接方式与内存管理方式**

- 基本结论

- 链接方式只强调链接的时机
- 不强制要求某一种内存管理方式

- 可搭配的装入情形

- 连续装入
- 离散装入
- 一次性装入
- 多次、按需装入
- 可搭配的内存管理方式
 - 固定分区分配
 - 动态分区分配
 - 基本分页
 - 基本分段
 - 段页式
 - 请求分页



请求分段



- 例：静态链接 + 基本分页
 - 装入前已经完成链接
 - 装入时再将完整程序分页
 - 各页可离散存放
 - 链接与分页并不冲突
- 例：运行时动态链接 + 请求分页
 - 只装入当前需要运行的页面
 - 执行到外部模块时再调入并链接
 - 多次装入不影响运行时动态链接

图灵社区会员 1931057098

• 段式最适合动态链接

- 基本结论
 - 运行时动态链接可以与多种内存管理方式搭配
 - 但段式管理最适合动态链接
- 原因
 - 动态链接通常以逻辑模块为单位
 - 函数、过程等天然构成独立逻辑段
 - 分段也以逻辑段为基本单位
 - 动态链接单位与内存分配单位一致
- 例：调用 `function A`
 - 程序只需要库文件中与 `function A` 有关的部分
 - 不需要调入整个库文件
 - 将 `function A` 所在逻辑段调入内存并链接即可
- 分页也能实现动态链接

- 但页不体现逻辑边界
 - 一个函数可能被拆分到多个页面
 - 需要分页、查页表并处理离散映射
 - 因而不如分段自然、方便
- 易错点
 - “段式最适合动态链接”不等于“只有段式能动态链接”

• 二、装入

- 装入解决的问题

- 将程序放入内存
- 处理程序地址
- 必要时完成逻辑地址到物理地址的转换
- 三种装入方式的区别
 - 核心在于：何时完成地址转换

- **逻辑地址与物理地址**

- 装入模块内部使用逻辑地址
 - 模块通常从逻辑地址 **0** 开始
 - 指令中的地址是相对于模块起点的地址
- 模块装入内存后具有物理起始地址 **R**
- 地址转换
 - 逻辑地址: **500**
 - 物理地址: **R + 500**
- 重定位
 - 将逻辑地址转换为物理地址

- **绝对装入**

- 适用环境
 - 单道程序环境
- 地址确定时机
 - 编译时
- 过程
 - 编译器事先知道程序将装入的内存位置
 - 编译后直接生成绝对地址
 - 装入程序按照指定地址装入
- 特点
 - 不需要进行地址转换

- 程序只能装入预先指定的位置
- 灵活性较差
- 限制
 - 多道程序并发时
 - 编译器无法提前确定程序将占用的内存位置
 - 因此不适合多道程序环境

• 静态重定位

- 又称
 - 可重定位装入方式
- 地址转换时机
 - 装入时
- 过程
 - 装入模块从逻辑地址 0 开始
 - 模块实际装入到物理起始地址 R
 - 装入程序修改模块中的有关地址
 - $\text{物理地址} = \text{逻辑地址} + R$

- 示例
 - 逻辑地址: 500
 - 物理地址: $R + 500$

- 特点
 - 程序可装入不同的内存位置
 - 地址在执行前一次性转换完成
 - 转换完成后地址被“写死”
 - 程序装入后不能再移动
- 适用场景
 - 固定分区分配
- 动态分区为何不适用
 - 动态分区容易产生外部碎片
 - 可能使用紧凑技术移动进程
 - 程序移动后，原先转换的物理地址将失效
- 判断关键
 - 只要程序装入后可能移动
 - 就不能使用静态重定位

• 动态重定位



程序执行前



后发先至公众号研究
永久微信1931037088

- 又称
 - 动态运行时装入方式
- 地址转换时机
 - 程序真正执行时
- 过程
 - 装入时保留逻辑地址
 - 执行前即使程序在内存中移动，也不修改逻辑地址
 - CPU 执行指令时实时进行地址转换
- 连续分配中的转换
 - 重定位寄存器保存程序的物理起始地址 **R**



点

- 点
 - 地址不会在装入时被写死
 - 允许程序在内存中移动
 - 程序移动后只需更新地址转换信息
- 适用场景
 - 动态分区分配
 - 基本分页
 - 基本分段
 - 请求分页
 - 请求分段
- 易错点
 - 动态重定位允许程序移动
 - 不等于使用动态重定位的程序必须发生移动

后发先至公众号研究
永久微信 1931057888



• 基本分页、基本分段为何属于动态重定位

- 基本分页、基本分段
 - 通常一次性装入
 - 不一定存在程序移动需求
- 仍属于动态重定位的原因
 - 地址转换发生在程序执行时
 - CPU 给出逻辑地址
 - MMU 通过页表或段表实时完成地址映射
 - 并非在装入时提前写死所有物理地址
- 判断标准
 - 不是看程序是否一定移动

- 而是看地址转换发生在装入时还是执行时

• 动态重定位的硬件支持

- 基本要求
 - 必须具有硬件地址转换机构
- 连续分配
 - 可使用重定位寄存器
 - 寄存器保存程序的物理起始地址
- 分页、分段
 - 不一定使用重定位寄存器
 - 分页通过页表完成地址转换
 - 分段通过段表完成地址转换



MMU 根据映射关系实时生成物理地址



- 严谨表述
 - 动态重定位必须有硬件支持
 - 但不一定必须使用“重定位寄存器”

• 三种装入方式对比

- 绝对装入
 - 地址确定时机：编译时
 - 地址转换：不需要
 - 能否移动：不能
 - 适用：单道程序环境
- 静态重定位
 - 地址确定时机：装入时、执行前
 - 地址转换：由装入程序一次性完成
 - 能否移动：转换后不能移动
 - 适用：固定分区分配
- 动态重定位
 - 地址确定时机：程序执行时
 - 地址转换：由硬件地址转换机构实时完成
 - 能否移动：允许移动
 - 适用：动态分区、分页、分段、请求分页等

后发先至公众号 1931057888

• 三、核心辨析

• 链接与装入的关注点

- 链接

- 关注：何时组合目标模块和外部模块
- 分类：静态链接、装入时动态链接、运行时动态链接
- 装入
 - 关注：何时完成地址转换
 - 分类：绝对装入、静态重定位、动态重定位

• 链接与内存管理

- 链接方式一般不限制内存管理方式
- 连续或离散、一次性或按需装入均可采用不同链接方式
- 但从便利性看
 - 段式最适合运行时动态链接

• 装入与内存管理

-  入方式与地址转换密切相关
- 绝对装入
 - 适用于单道程序环境
- 静态重定位
 - 适用于装入后不移动的固定分区
- 动态重定位
 - 适用于存在移动需求的管理方式
 - 也适用于执行时动态查表转换地址的分页、分段方式



• 高频易错点

- 动态链接通常默认指运行时动态链接
- 段式最适合动态链接，但分页也能实现动态链接
- 静态重定位完成后，程序不能在内存中移动
- 动态重定位允许移动，但不要求程序必须移动
- 基本分页、基本分段也属于动态重定位
- 动态重定位必须有硬件地址转换机构，但不一定使用重定位寄存器